

V-Decent Application Development Guide (en, V3_0)

Document Version: 3.0

Target Platform: V-Decent Distributed Node Cluster (Coolify Orchestration)

Line Spacing: 1.15

1. Introduction

This document serves as the standardized engineering guide for developers building, packaging, and deploying multi-tier applications across the V-Decent distributed network infrastructure. By optimizing container networking topologies, the platform ensures secure, zero-trust host execution alongside high density, scaling up to 20 production application stacks per bare-metal or cloud host instance without system resource starvation.

2. Development Lifecycle & Responsibilities

Application engineering teams maintain complete ownership over internal software design, local runtime composition, and environment isolation logic. Platform operations teams manage high-availability host boundaries, global load balancing, and backing storage networks. Handover into staging or production systems is driven automatically via declarative configuration manifests managed by Coolify control panels.

2.1 The Deployment Flow

- **Design & Coding:** Developers build the application according to V-Decent standards (Docker-based).
- **Local Verification:** Developers **MUST** test the application on their local server (e.g., using the sample repository provided) to ensure it functions correctly as a containerized service.
- **Customer Approval:** The Application Owner (customer) reviews the verified application.
- **Handover:** Once approved, the Developers **MUST** register the target app via V-Decent Application Manager (<https://am-dev.v-decent.org>).
 - Developer will register the application to V-Decent Application Manager, and submit request for deploy
 - There are certain requirements for the application to be compatible with V-Decent ecosystem and the developers **MUST** follow the details explained later in this document
 - Upon the approval by V-Decent Operator, the application will be deployed to the most appropriate (evaluated by Selection Score automatically) node.
- **Production Deployment:** V-Decent Operators perform the actual deployment to production nodes via the Coolify orchestration layer.

V-Decent Application Development & Deployment Lifecycle



3. Supported Application Types

Currently, the V-Decent ecosystem is optimized for stateless applications. These applications do not store persistent data, within the container's file system neither externally.

- **Stateless Compute Services:** API gateways, frontend web applications, background worker microservices, and chronological task processors.
- **Stateful Storage Services:** Relational and non-relational database instances, persistent data caching nodes, and structured file queues utilizing platform-managed underlying storage volumes.

3.1 Stateless Applications (Currently Supported)

Currently, the V-Decent ecosystem is optimized for stateless applications. These applications do not store persistent data, within the container's file system neither externally.

3.2 Stateful Applications with External Data (Currently Supported)

Support for stateful applications with data stored external, not in the V-Decent ecosystem, is still supported. Since, the external storage will manage the data, as long as the application manages the reference to this external storage by using Environment Value, it is not a problem. Thus, the application should:

- Manage the application data in the external storage (Ex. AWS S3, Wasabi, ...)
- Use Environment Value to point to the external storage

3.3 Stateful Applications with Data (Supported with

Limitation)

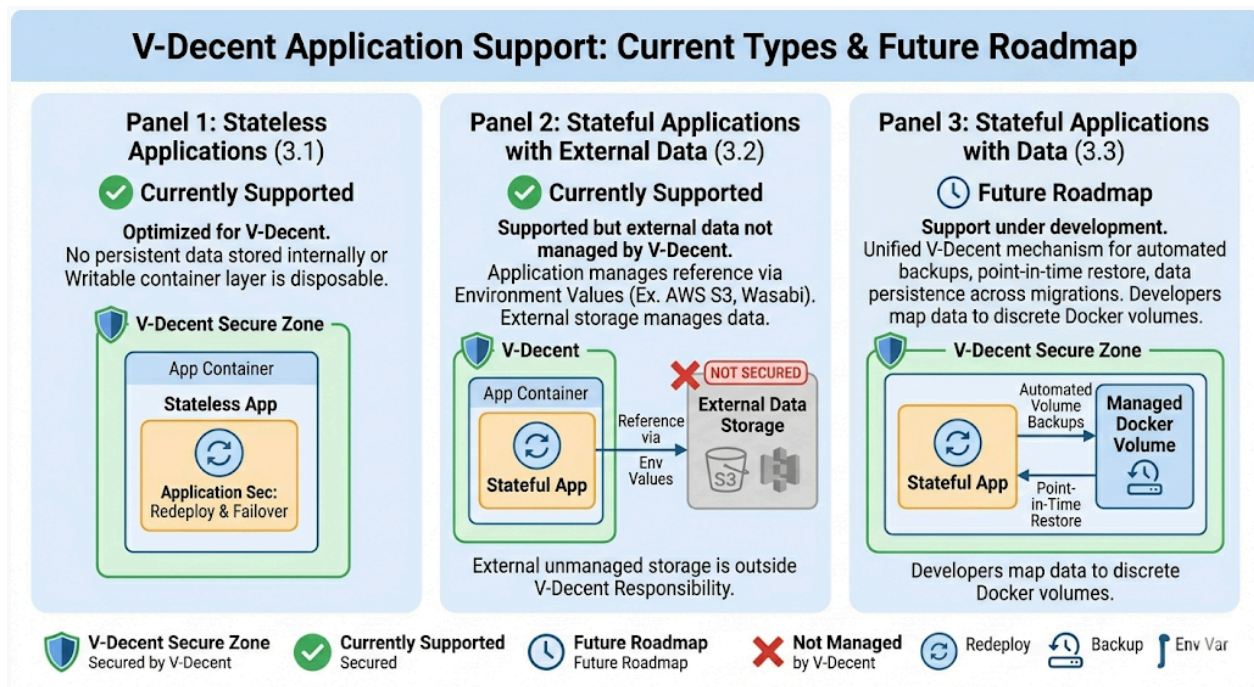
Support for stateful applications with external data backup:

- Automated backups of the application, including the database, is supported
- The database data is not guaranteed
- With the application packed with the backup sidecar container, database data can be backup and restore from external storage
- Data backup and restore to/from Google Drive is supported, with point-in-time restore capability and data persistence across node migration

The V-Decent platform will soon introduce a unified mechanism for:

- Automated backups of application data volumes.
- Point-in-time restore capabilities.
- Data persistence across node migrations.

Developers planning stateful apps should ensure their data is mapped to discrete Docker volumes to prepare for this upcoming feature.



4. Technical Requirements for V-Decent Compatibility

To operate seamlessly inside the V-Decent multi-tenant environment, all applications must completely decouple public routing infrastructure from internal workload communication. The underlying Docker daemon dynamically manages subnet pools behind the scenes. Developers must follow standard, non-intrusive container formatting specifications to avoid proxy routing faults and upstream communication drops.

4.1 Docker Configuration Specification

The following parameters are mandatory for any application stack submitted for platform deployment. Compliance ensures that individual containers are correctly registered by automated proxy components while maintaining zero host-level port exposures.

Requirement	Technical Specification & Description
No Host Port Mapping	Do not map, expose, or bind physical port configurations directly to the host network interface (e.g., avoid ports: "8080:80"). Direct bindings create network conflicts across multi-tenant nodes. All inbound ingress routing paths are dynamically managed and isolated by Coolify proxy instances.
Port Declaration	Use the expose array parameter to identify internal networking endpoints. This alerts upstream container bridges where application traffic is listening.
Native Network Isolation (NOT SUPPORTED)	Due to a limitation of Coolify/Traefik, defining a network in the docker compose MAY NOT work as expected. So, we STRONGLY RECOMMEND NOT TO USE IT. (To be supported) Define internal-only software bridge networks (e.g., external-tier, internal-tier) within the application manifest. Do not hardcode reference configurations to global external networks like vdecent-ingress. This prevents conflicts with runtime orchestration engines.
Environment Variables	Isolate all secrets, connection tokens, and environment parameters into standard .env configurations. Include a fully documented, unpopulated .env.example reference profile inside the root repository.

Requirement	Technical Specification & Description
Liveness & Readiness Probes	Integrate explicit Docker engine healthcheck routines for all downstream data storage systems and caching daemons to confirm initialization status before compute workers begin boot cycles.

4.2 Environment Variables

For the Application Variables, please provide the .env content or the file that is to be used by the application for production. The sample repository includes the example of the Environment Variables:

<https://github.com/luizcarloskazuyukifukaya/stateless-app-for-vdecent/blob/main/.env.example>

```
# App Configuration
PORT=8081
SITE_NAME=Timezone Web App
PRIMARY_COLOR=#1a73e8
```

Developers are to hand over these Environment Variables (**with the actual value to be used for production**) as a part of the application information when submitting the application for production.

5. Reference Implementation

5.1 Sample Public Repository

For a practical example of a verified stateless application setup, refer to the official V-Decent sample repository:

- Simple Stateless App:
<https://github.com/luizcarloskazuyukifukaya/stateless-app-for-vdecent>
- Simple Stateful App with External Backup:
<https://github.com/luizcarloskazuyukifukaya/stateful-app-with-external-backup-for-vdecent>

(NOTE) This sample introduces the sidecar container that performs the backup and restore to/from Google Drive.

5.2 Reference Implementation Setup

The following standardized blueprint demonstrates a highly resilient, two-tier architecture comprising a web runtime environment and an isolated database layer. This structure utilizes

isolated application bridges that draw subnets automatically from expanded host pools while allowing Coolify to cleanly map its public ingress hooks at runtime.

```
services:
  app:
    build: .
    restart: always
    environment:
      - DATABASE_URL=postgres://user:password@db:5432/activitylog
    expose:
      - "80"
    depends_on:
      db:
        condition: service_healthy
# DO NOT DEFINE NETWORK
#   networks:
#     - vdecent-network # vdecent network for all containers
#     - external-tier # Internal web-tier isolation
labels:
  - "coolify.managed=true" # Notify Coolify to use this network

  db:
    image: postgres:16-alpine
    restart: always
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=activitylog
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d activitylog"]
      interval: 5s
      timeout: 5s
      retries: 10
# DO NOT DEFINE NETWORK
#   networks:
#     - vdecent-network # vdecent network for all containers
#     - internal-tier # Database remains completely isolated from
web vectors

# DO NOT DEFINE NETWORK
#networks:
# vdecent-network: # bridge network for v-decent
#   driver: bridge
# external-tier: # Dynamically allocated bridge network for web
ingress
#   driver: bridge
# internal-tier: # Dynamically allocated bridge network for isolated
```

```
database traffic
#   driver: bridge

volumes:
  postgres_data:
```

6. GitHub Repository

6.1 Public Repository

If your repository is set as Public, then there is no settings required on the Git Provider (like GitHub, GitLab, etc .) side, and you should:

- 1) Provide the following information to V-Decent operators:
 - a) Repository URL (HTTPS)
 - b) Environment Variables

Common pitfalls

- Wrong repo URL format (HTTPS mismatch)

6.1 Private Repository

If your repository is set as Private, then you need to share it with V-Decent operators so they can properly access it for deployment.

To deploy an application from a Git provider (like GitHub, GitLab, etc.) **private repository**, V-Decent Application Manager needs to be authorized to access it. For that, Developers MUST:

- Associate the Application Manager login user with the Git provider account
- Acquire the Public Key assigned for the developer's Git provider account from the Application Manager
- Configure the Git provider with the **Public Key issued by Application Manager**

7. Application Deployment in V-Decent

7.1 Conditions

The Application deployment is conducted using V-Decenter Application Manager, and it realizes the easy and user-friendly operation flow. Here are the basics for conducting application deployment using the Application Manager: A Developer MUST

- have access to V-Decent Application Manager (login user and password required)
- Belong to a group (aka. Developer Group)
- Have a GitHub account
- Comply to [Technical Requirements for V-Decent Compatibility](#) written above (the repository should be docker compose based)

7.2 Application Registry

The Developer MUST register the application using Application Manager. The following are the steps on how to register the application:

- 1) Login in to V-Decent Application Manager
 - Access <https://am-dev.v-decent.org>
 - Login with your account
 - Select “Applications” - “Register Application”
- 2) Configure the application
 - Developers need to provide Identity and Ownership, Application Details:

Register New Application
Configure your application deployment settings

IDENTITY & OWNERSHIP

DEVELOPER GROUP: V-Decent Developers

GITHUB ACCOUNT / ORG: luizcarloskazuyukifukaya

APPLICATION DETAILS

APPLICATION NAME: V-Decent Sample

PRIMARY SHORTNAME: sample

PRIMARY FQDN: sample.v-decent.org

CATEGORY: Small (Light resources)

Editable for custom domains. Default: shortname.v-decent.org

- Developers need to provide Deployment Source (for Public Repository):

 **DEPLOYMENT SOURCE**

GITHUB REPOSITORY URL

`https://github.com/luizcarloskazuyukifukaya/stateful-app-for-vdecent`

REPOSITORY VISIBILITY BRANCH

 Public Repository main  Analyze Manifest

 **SECURITY & CONFIGURATION**

ENVIRONMENT VARIABLES ⓘ

```
API_KEY=secret_value DATABASE_URL=postgres://...
```

- Developers need to provide Deployment Source (for Private Repository):

 **DEPLOYMENT SOURCE**

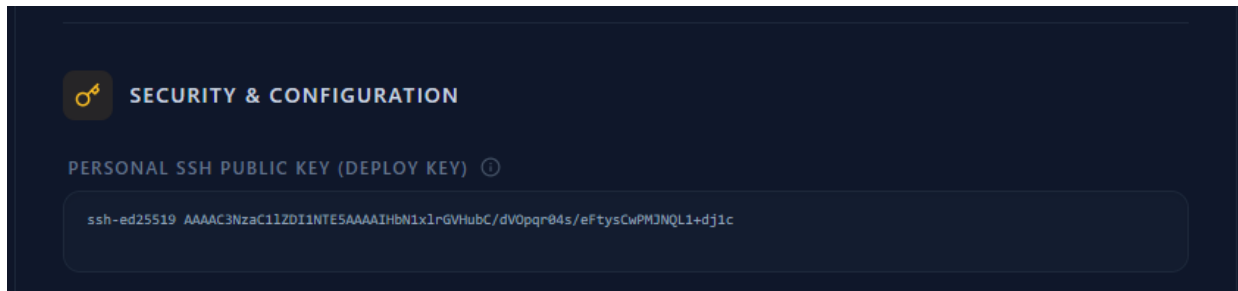
GITHUB REPOSITORY URL

`https://github.com/luizcarloskazuyukifukaya/stateful-app-for-vdecent`

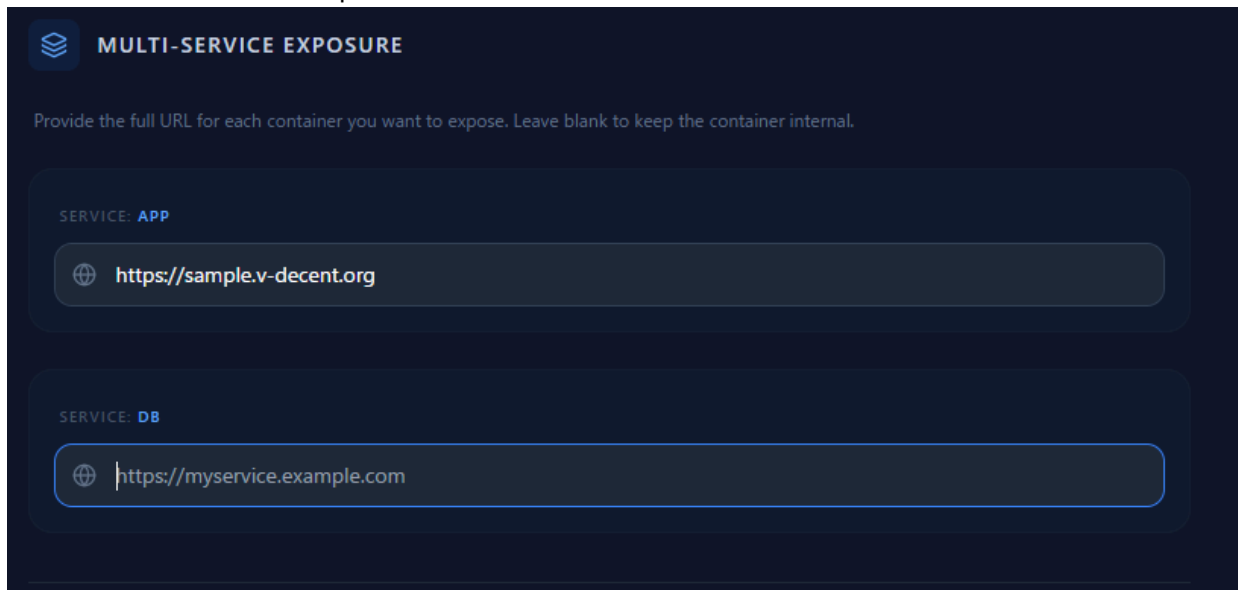
REPOSITORY VISIBILITY BRANCH

 Private Repository main  Analyze Manifest

After specifying the Git repository URL, selecting “Private Repository, and specifying the Branch name, you **SHOULD** click “**Analyze Manifest**”



- Application Manager will try to access the private repository with the Public Key configured on your Git provider account/repository, and if it is permitted access, it will identify the manifest file (docker-compose.yaml) to capture the application docker composer structure
- (NOTE) The Public Key to use is shown when selecting “Private Repository”
- Based on the docker compose file content, Application Manager will ask the Developer to decide which container to expose:



```

services:
  app:
    build: .
    restart: always
    environment:
      - DATABASE_URL=postgres://user:password@db:5432/activitylog
    depends_on:
      db:
        condition: service_healthy
    networks:
      - activity-network
    labels:
      - "coolify.managed=true"
  db:
    image: postgres:16-alpine

```

```

restart: always
environment:
  - POSTGRES_USER=user
  - POSTGRES_PASSWORD=password
  - POSTGRES_DB=activitylog
volumes:
  - postgres_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U user -d activitylog"]
  interval: 5s
  timeout: 5s
  retries: 10
networks:
  - activity-network

networks:
  activity-network:
    driver: bridge

volumes:
  postgres_data:

```

- (NOTE) For this docker-compose.yml example, app container and db container are in the manifest file, so this means 2 containers are to be deployed. The developers need to specify which ones to expose via Application Manager with the specific URL. **If the container is NOT to be exposed, the corresponding URL field MUST be empty.**
- Specify the environment variables

ENVIRONMENT VARIABLES ⓘ

```

# App Configuration
PORT=8081

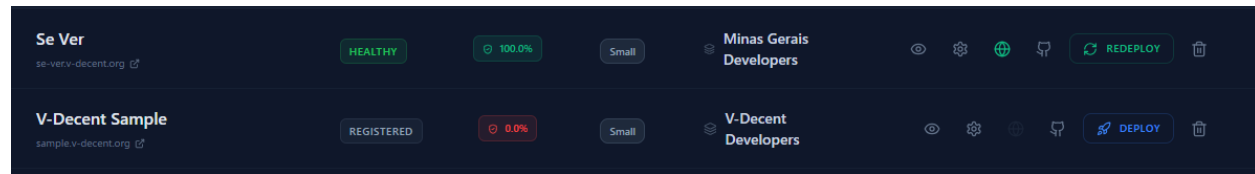
```

- Select “Register Application” to finalize the registration

7.3 Application Deployment

Once the application is registered, all the developers need is to select “Deploy” from the Application Manager.

- 1) Locate the target application to deploy in the Applications
 - Select “Applications”



“V-Decent Sample” is “REGISTERED” and the “DEPLOY” button is available

- 2) Click “DEPLOY” to initiate the deployment
- 3) Wait for the deployment to complete
 - When the deployment completed successfully, the application status should change to “HEALTHY”
 - If failed, then the status will be “UNHEALTHY”, and you should check the System Logs for details.

Common pitfalls

- The repository is not with the expected **docker-compose.yaml** file (The docker-compose file name matters!!!)
- The repository is private and the Public Key is not configured.
- The Public Key is wrong. Make sure you copy the right value shown in the Application Manager Application Registry screen.
- The URL specified is still not supported by V-Decent Application Manager - note that as of this version (V2.0), the custom domains are not supported. Only *.v-decent.org can be used for application URLs.